

NAME : SHAH FAISAL

ROLL NO : 23I-0058

SECTION : BSE – 4A

OS LAB 06

BSSE SPRING 2026

LAB 06

Tasks:

I. Extend the example 2 to allow multiple messages to be sent back and forth between the parent

and child processes.

Implement a program where the parent process sends a series of three messages to the child process through one pipe.

The child process should read each message, print it, and then send a response message back to

the parent process through another pipe.

Both processes should handle multiple messages in a loop and ensure that each message is correctly sent and received. [Estimated Time: 30 mins]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int pipe1[2]; // Parent to child pipe
```

```
    int pipe2[2]; // Child to parent pipe
```

```
    pid_t pid;
```

```
    char buf[100];
```

```
    char *parent_messages[] = {
```

```
        "message 1",
```

```
        "message 2",
```

```
        "message 3"
```

```
};
```

```
char *child_responses[] = {
    "reply 1",
    "reply 2",
    "reply 3"
};

int num_messages = 3;

if (pipe(pipe1) == -1) {
    printf("Error creating pipe1\n");
    exit(1);
}

if (pipe(pipe2) == -1) {
    printf("Error creating pipe2\n");
    exit(2);
}

pid = fork();

if (pid == -1) {
    printf("Error creating process\n");
    exit(3);
}

if (pid == 0) {
    close(pipe1[1]); // Close write end of pipe1
    close(pipe2[0]); // Close read end of pipe2
    for (int i = 0; i < num_messages; i++) {
        // Read message from parent
        ssize_t bytesRead = read(pipe1[0], buf, sizeof(buf) - 1);
        if (bytesRead == -1) {
            printf("Error reading from pipe1\n");
            exit(4);
        }
    }
}
```

```
}  
  
buf[bytesRead] = '\0';  
printf("Child received %s\n", buf);  
  
// Send response to parent  
ssize_t bytesWritten = write(pipe2[1], child_responses[i],  
                             strlen(child_responses[i]) + 1);  
  
if (bytesWritten == -1) {  
    printf("Error writing to pipe2\n");  
    exit(5);  
}  
}  
  
close(pipe1[0]);  
close(pipe2[1]);  
exit(0);  
  
} else { // Parent process  
  
    close(pipe1[0]);  
    close(pipe2[1]);  
  
    for (int i = 0; i < num_messages; i++) {  
  
        // Send message to child  
        ssize_t bytesWritten = write(pipe1[1], parent_messages[i],  
                                     strlen(parent_messages[i]) + 1);  
  
        if (bytesWritten == -1) {  
            printf("Error writing to pipe1\n");  
            exit(6);  
        }  
  
        ssize_t bytesRead = read(pipe2[0], buf, sizeof(buf) - 1);
```

```
if (bytesRead == -1) {  
    printf("Error reading from pipe2\n");  
    exit(7);  
}  
buf[bytesRead] = '\0';  
printf("parent received %s\n", buf);  
}  
close(pipe1[1]);  
close(pipe2[0]);  
wait(NULL);  
exit(0);  
}  
}
```



shah@DESKTOP-02Q2MPC: ~

```
shah@DESKTOP-02Q2MPC:~$ gcc lab6_q1.c -o lab6  
shah@DESKTOP-02Q2MPC:~$ ./lab6  
Child received message 1  
parent received reply 1  
Child received message 2  
parent received reply 2  
Child received message 3  
parent received reply 3  
shah@DESKTOP-02Q2MPC:~$
```

II. Create multiple child processes, each communicating with the parent through shared pipes.

Implement a parent process that spawns three child processes.

Each child process should perform some simple computation (e.g., compute the square of a number) and send the result to the parent through a pipe.

The parent process should collect results from all child processes and print them.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#define NUM_CHILDREN 3

int main() {

    int pipes[NUM_CHILDREN][2];

    pid_t pids[NUM_CHILDREN];

    int numbers[NUM_CHILDREN] = {2, 3, 4};

    int results[NUM_CHILDREN];

    for (int i = 0; i < NUM_CHILDREN; i++) {

        if (pipe(pipes[i]) == -1) {

            printf("Error creating pipe for child %d\n", i + 1);

            exit(1);

        }

    }

    for (int i = 0; i < NUM_CHILDREN; i++) {

        pids[i] = fork();

        if (pids[i] == -1) {

            printf("Error creating child process %d\n", i + 1);

            exit(2);
```

```
}
```

```
if (pids[i] == 0) {  
    close(pipes[i][0]);  
    int result = numbers[i] * numbers[i];  
    write(pipes[i][1], &result, sizeof(result));  
    close(pipes[i][1]);  
    exit(0);  
}  
}  
for (int i = 0; i < NUM_CHILDREN; i++) {  
    close(pipes[i][1]);  
    read(pipes[i][0], &results[i], sizeof(results[i]));  
    close(pipes[i][0]);  
    printf("Parent received: Child %d result\n", i + 1);  
}  
for (int i = 0; i < NUM_CHILDREN; i++) {  
    wait(NULL);  
}  
return 0;
```

}

```
shah@DESKTOP-02Q2MPC:~$ nano lab6_q2.c
shah@DESKTOP-02Q2MPC:~$ gcc lab6_q2.c -o lab5
shah@DESKTOP-02Q2MPC:~$ ./lab5
Parent received: Child 1 result
Parent received: Child 2 result
Parent received: Child 3 result
shah@DESKTOP-02Q2MPC:~$
```